# Laboratório Funcional · Aula 12

Paulo Henrique Malavasi · RA 24368074-2 · Turma C
Gerado em 17/10/2026, 10:06:59 · tempo registrado no cronômetro: 0:00

## Preparação: Programar com funções matemáticas

**def area(r): return 3.14159 * r * r**

Pura

**def agora(): return datetime.now()**

Impura

**def total(lista): return sum(lista)**

Pura

**def registra(msg): print(msg)**

Impura

**def sorteia(): return random.randint(1, 6)**

Impura

**def poe(lista, x): lista.append(x); return lista**

Impura

**0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.**

A função agora() é impura porque depende do horário atual, podendo retornar valores diferentes a cada execução. Já a função registra(msg) é impura porque utiliza print(), causando um efeito colateral ao exibir uma mensagem na tela.

## Prática 1: Funções puras e formas funcionais

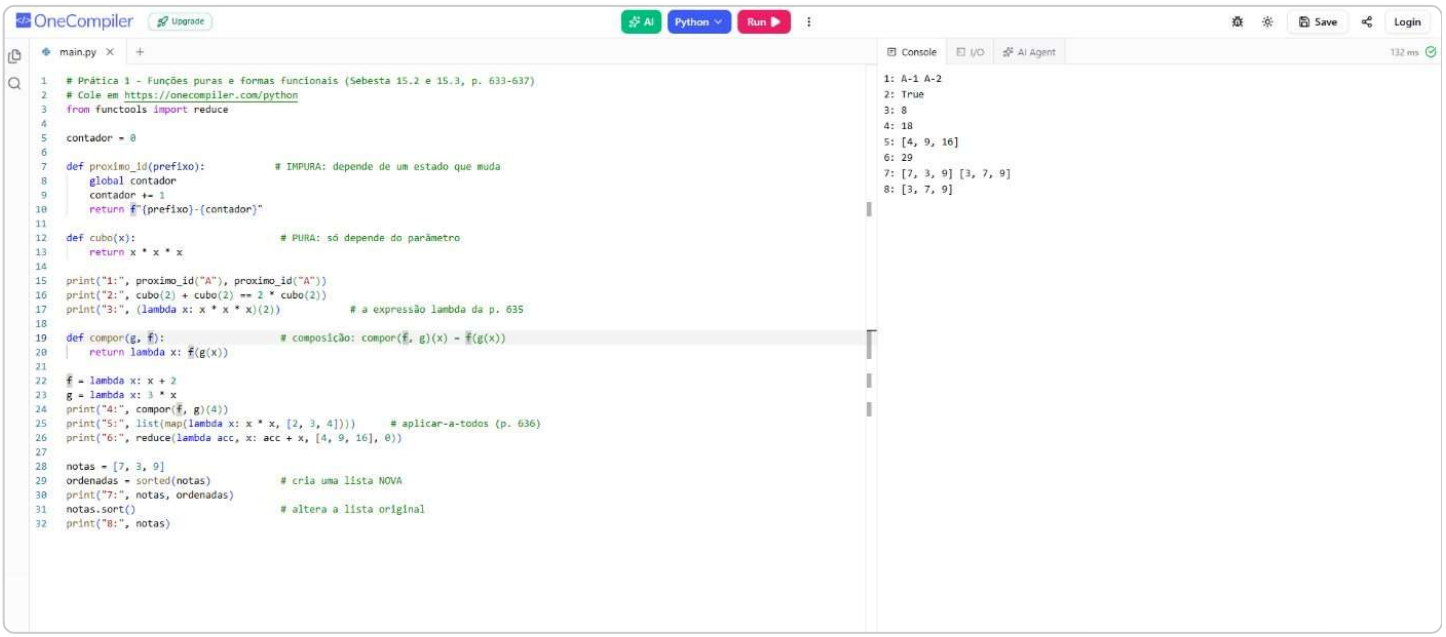
**Previsão: O que as linhas 1, 4 e 7 vão imprimir?**

*não respondido*

**Saídas (original e com compor(g, f))**

*não respondido*




**1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?**

função proximo_id("A") retorna valores diferentes porque utiliza um contador global que aumenta a cada chamada. Isso dificulta os testes, pois o resultado depende das execuções anteriores, e não apenas da entrada.

**1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.**

compor(f, g)(4) = 14, pois 4 × 3 = 12 e 12 + 2 = 14.
compor(g, f)(4) = 18, pois 4 + 2 = 6 e 6 × 3 = 18.

**1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?**

sorted(notas) cria uma nova lista ordenada sem alterar a original, enquanto notas.sort() modifica a própria lista. O sorted() segue melhor o estilo funcional porque evita alterar os dados originais e causar efeitos colaterais.

## Prática 2: Scheme: tudo é lista

**Previsão: (car '(A B C))**

*não respondido*

**Previsão: (cdr '(A B C))**

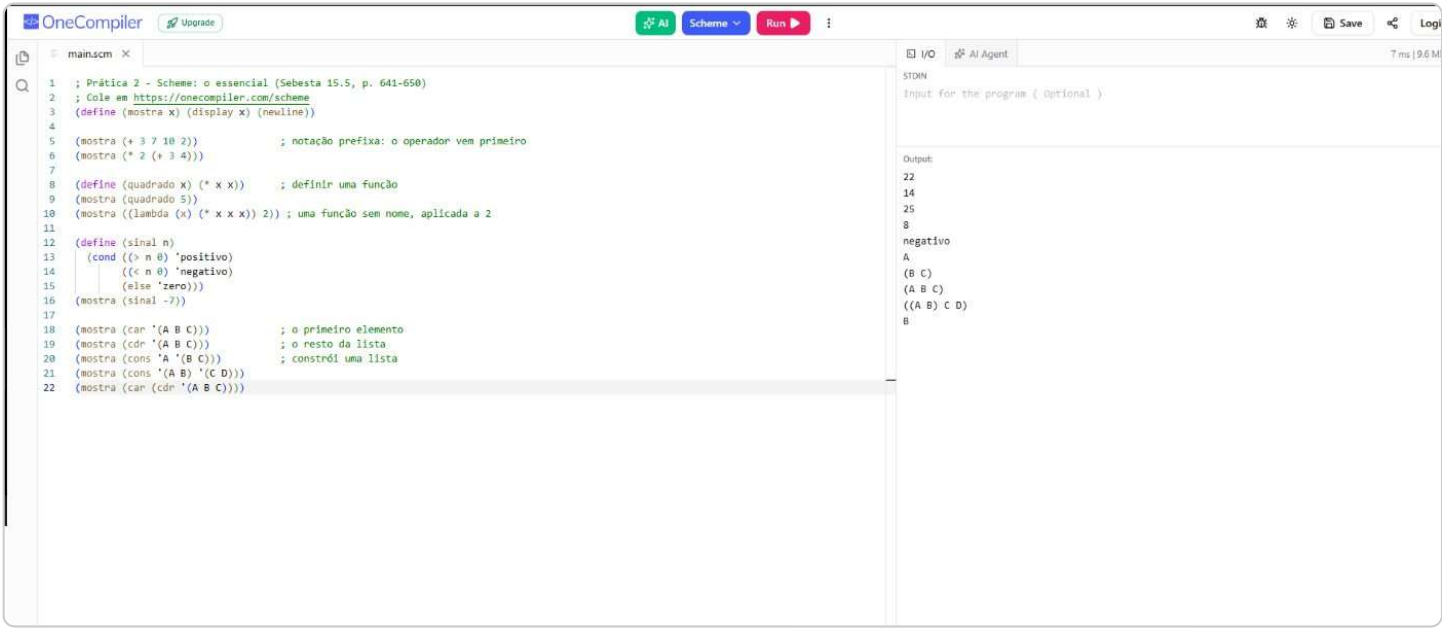
*não respondido*

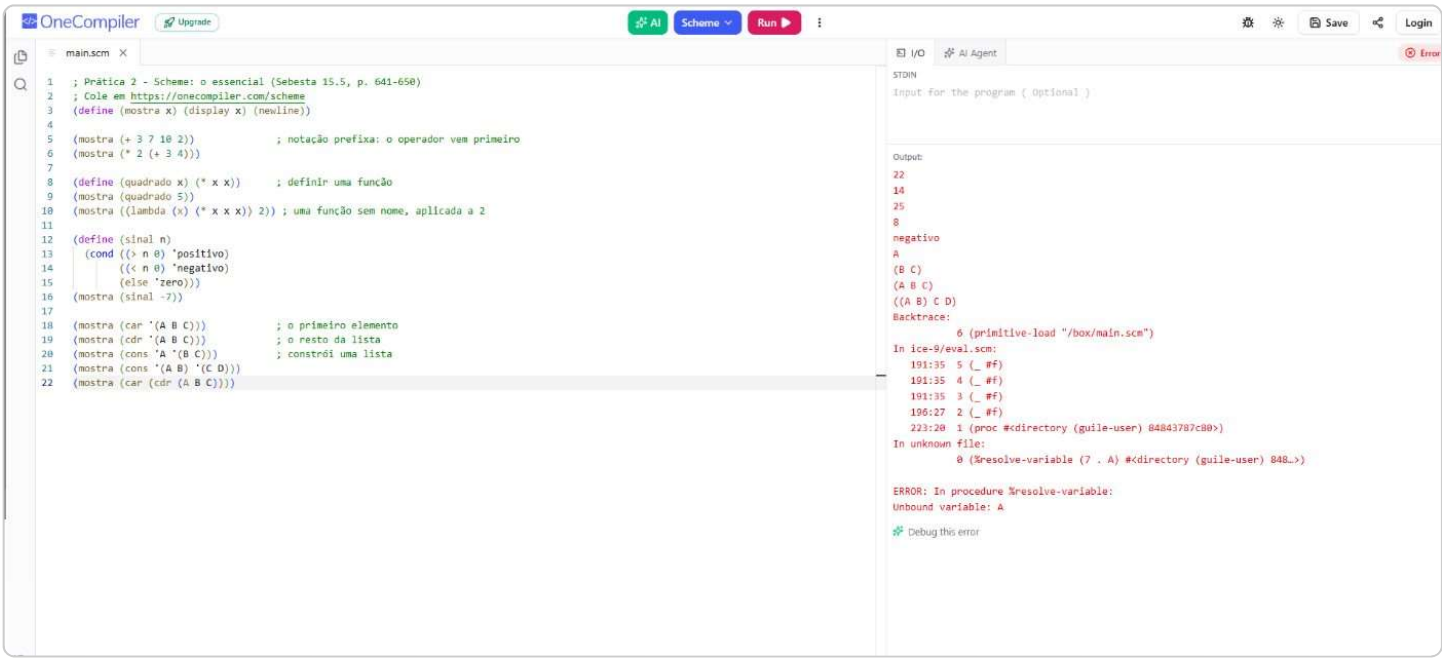
**Previsão: (cons '(A B) '(C D))**

*não respondido*

**Saídas (original e sem o apóstrofo)**

*não respondido*




**2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?**

O apóstrofo (') serve para indicar que a lista deve ser tratada como dados, sem executar seu conteúdo. Sem ele, o Scheme tentou executar A como uma função, usando B e C como argumentos, causando um erro.

**2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.**

```
(car (cdr (cdr '(A B C))))
```

C

## Prática 3: Recursão e recursão em cauda

**O que vi no passo a passo**

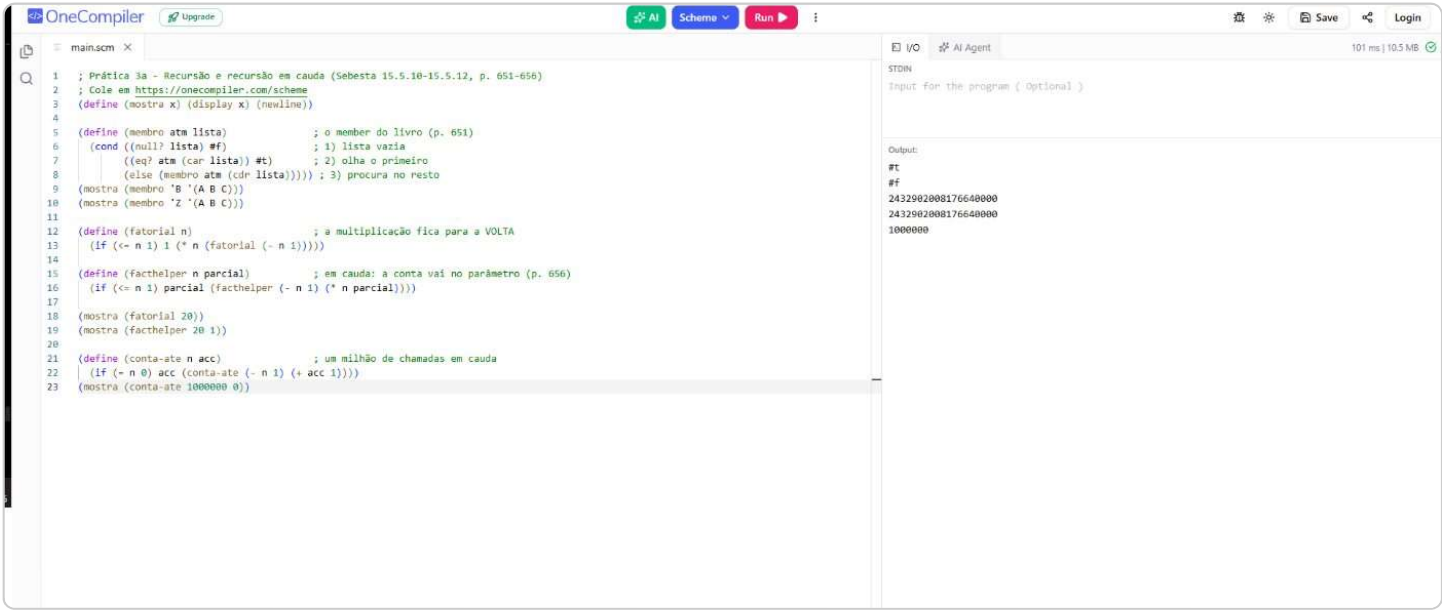
*não respondido*

**Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?**

*não respondido*

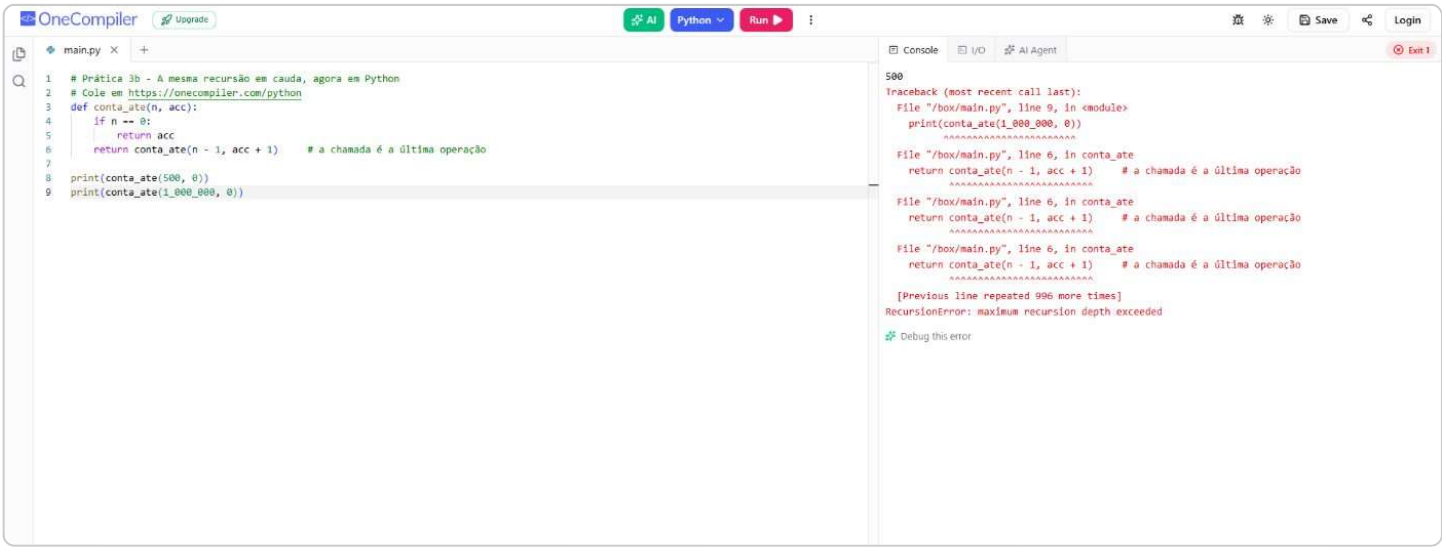
**Saída do Scheme**

*não respondido*




**Saída do Python (as últimas linhas bastam)**

*não respondido*




**3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?**

Na recursão comum, cresce a pilha de chamadas, pois cada função precisa esperar o retorno da próxima. Na recursão em cauda, isso não acontece quando há otimização, pois a chamada anterior pode ser substituída pela

próxima.

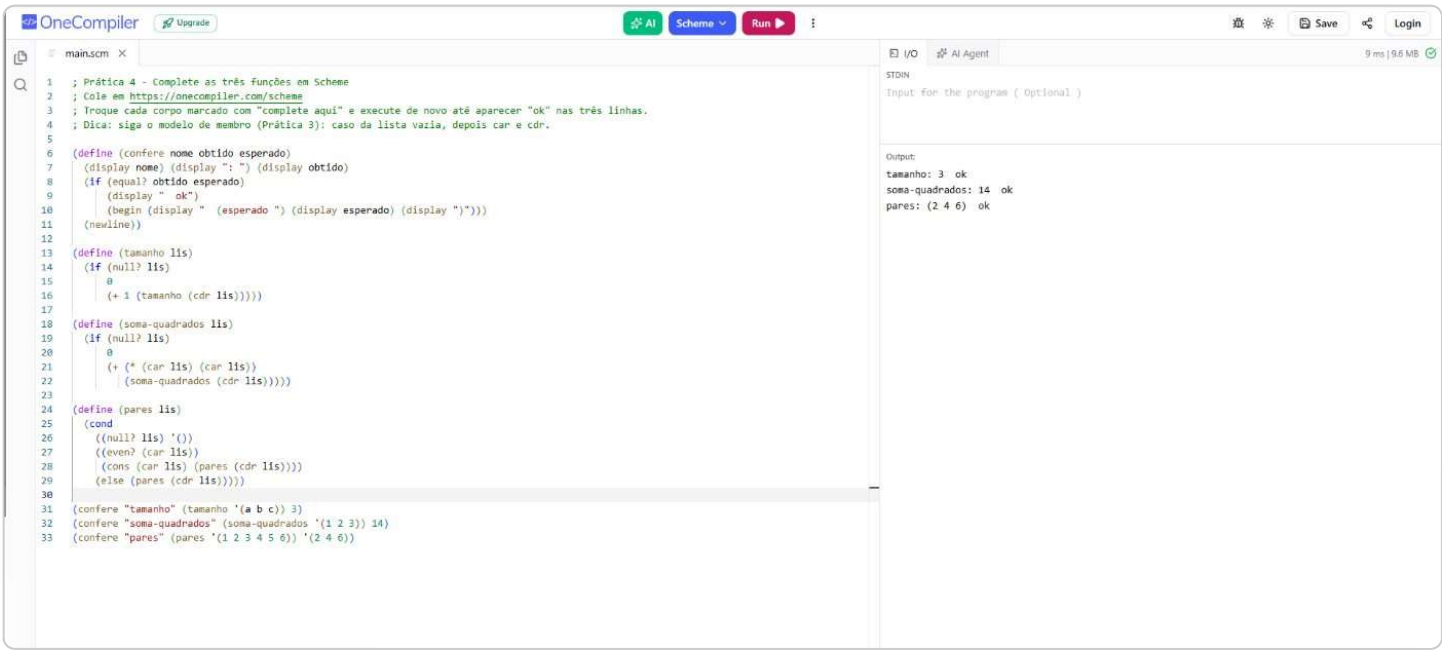
**3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)**

O Scheme utiliza a otimização de recursão em cauda, permitindo executar um milhão de chamadas sem aumentar a pilha. Já o Python não realiza essa otimização e possui um limite de recursão, causando o RecursionError quando esse limite é ultrapassado.

## Prática 4: Mão na massa: três funções sobre listas

**Saída final (com os três ok)**

*não respondido*




**4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).**

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond
    ((null? lis) '())
    ((even? (car lis))
     (cons (car lis) (pares (cdr lis))))
    (else (pares (cdr lis)))))
```

**4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?**

A parte mais difícil foi montar a recursão para filtrar os números pares. Nas três funções, usei null? para verificar se a lista estava vazia e cdr para percorrer os elementos. Usei car na soma dos quadrados e na função de pares para acessar o primeiro elemento da lista.

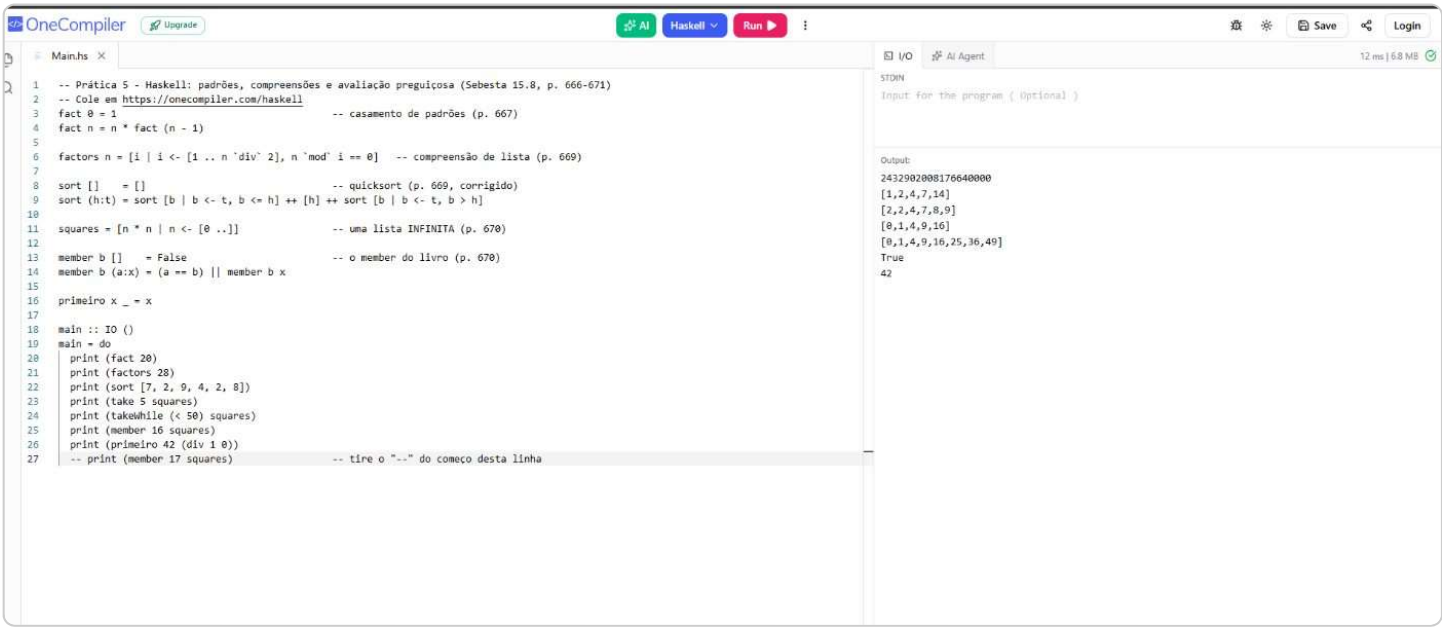
## Prática 5: Haskell: padrões, compreensões e preguiça

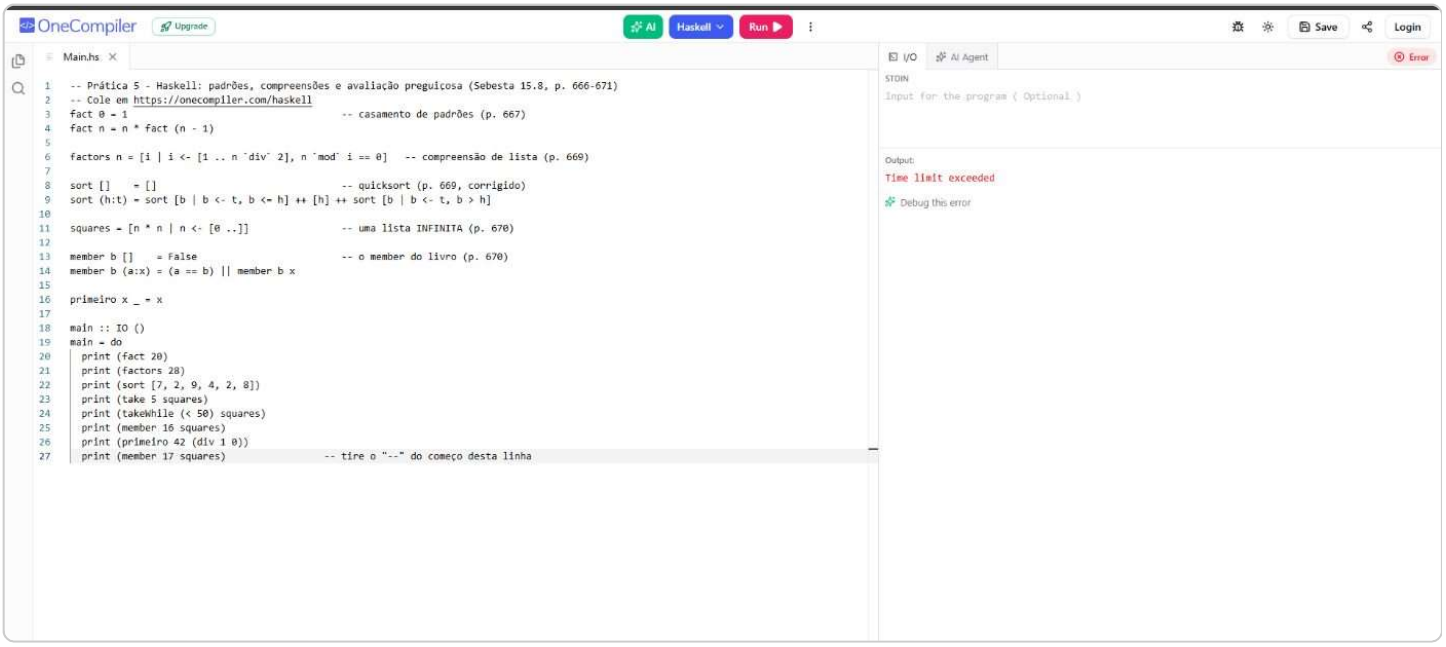
**Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?**

*não respondido*

**Saídas (original e com member 17)**

*não respondido*




**5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?**

O Haskell utiliza avaliação preguiçosa, calculando apenas os elementos necessários da lista infinita. A função primeiro 42 (div 1 0) não deu erro porque retorna apenas o primeiro argumento, sem precisar avaliar a divisão por zero.

**5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)**

member 17 squares nunca termina porque 17 não é um quadrado perfeito, então a função continua procurando na lista infinita. Já member 16 squares termina porque encontra o número 16.
Para corrigir, basta interromper a busca quando o elemento atual ultrapassar o valor procurado, já que a lista está em ordem crescente.

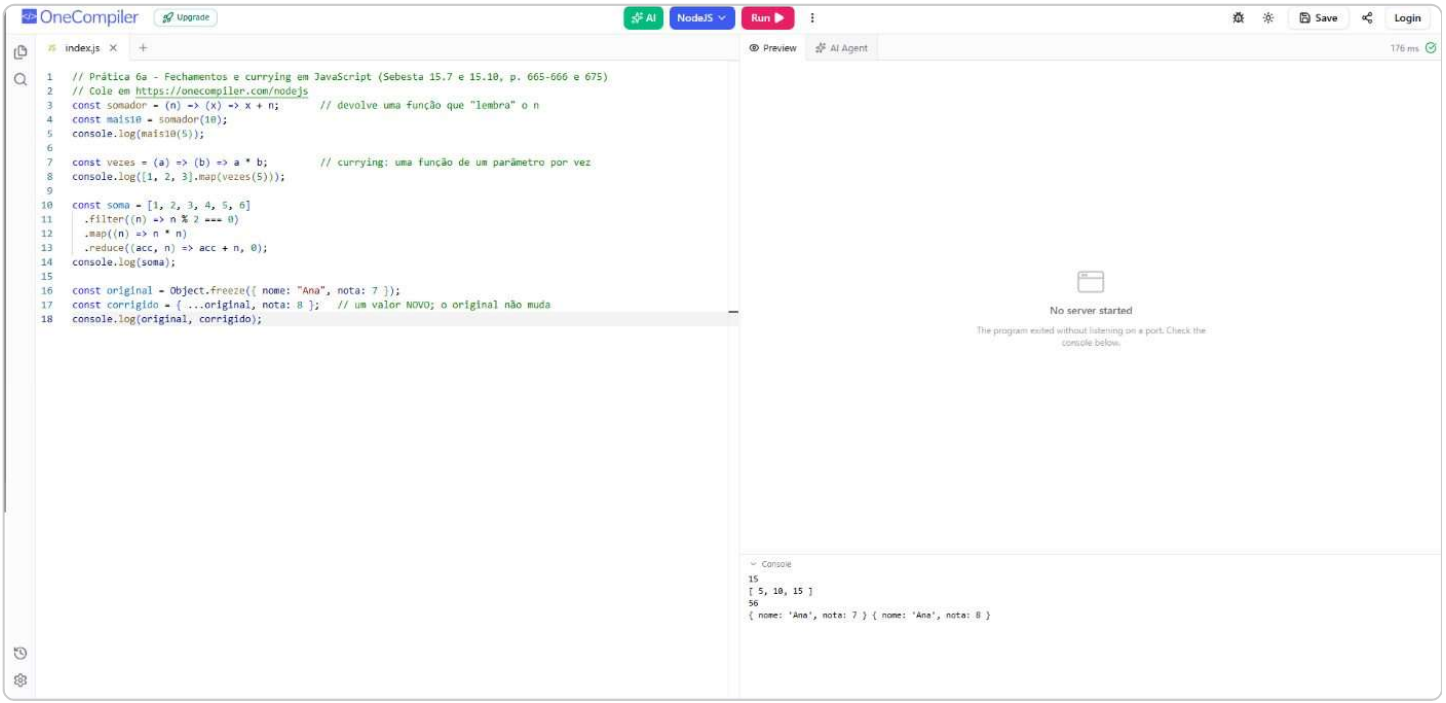
## Prática 6: Você já usa funcional

**Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?**

*não respondido*

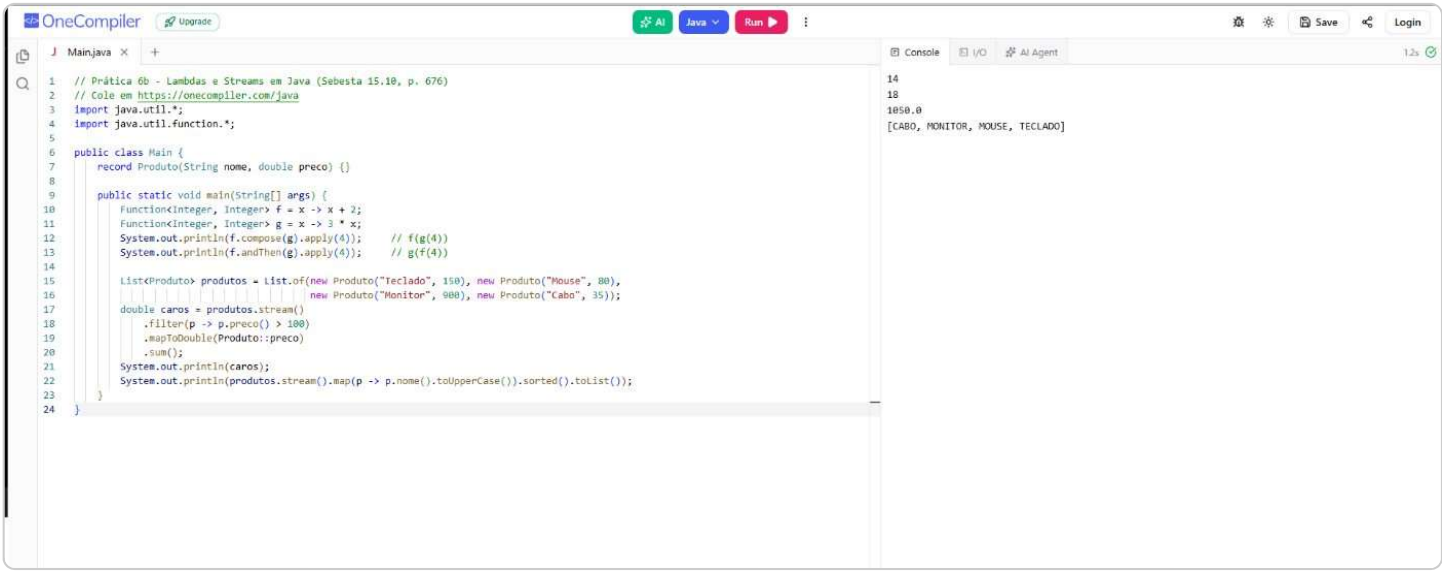
**Saída do JavaScript**

*não respondido*




**Saída do Java**

*não respondido*




**6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?**

Isso ocorre por causa do closure (fechamento). A função mais10 mantém o valor 10 armazenado no ambiente em que foi criada, mesmo depois que somador(10) terminou sua execução.

**6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?**

Os resultados são diferentes porque a ordem de execução das funções muda. f.compose(g).apply(4) retorna 14, pois executa g antes de f. Já f.andThen(g).apply(4) retorna 18, pois executa f antes de g. O compose corresponde ao compor(f, g) da Prática 1.

## Prática 7: O laço vira pipeline

**7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
```

**7.2 Segundo exercício escolhido: cole o seu código reescrito.**

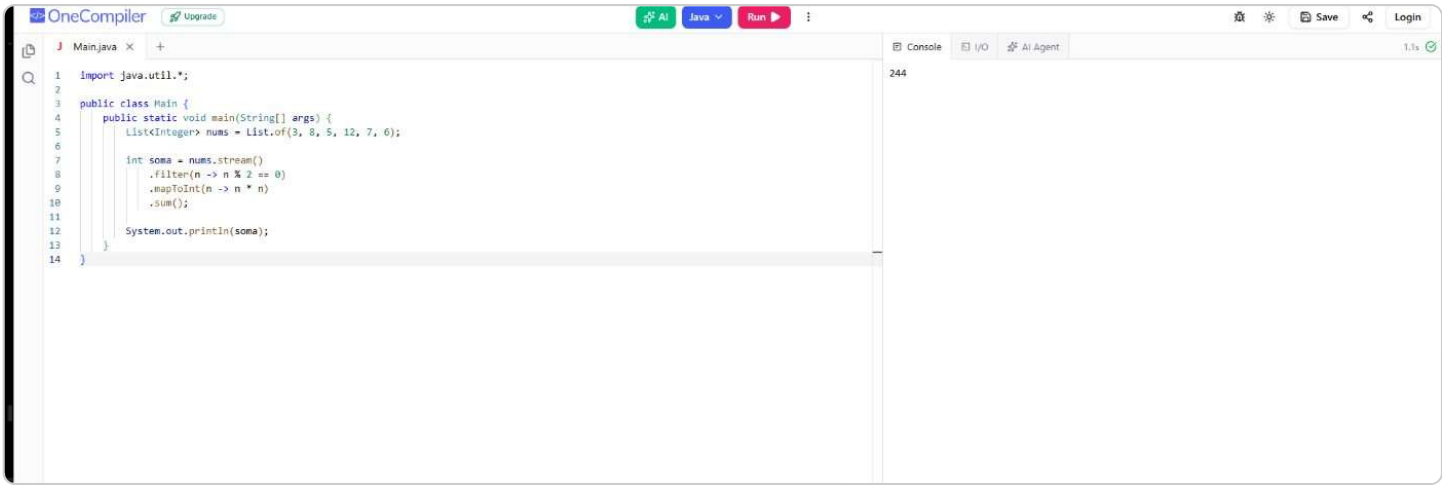
```
precos = [50.0, 120.0, 80.0, 200.0]

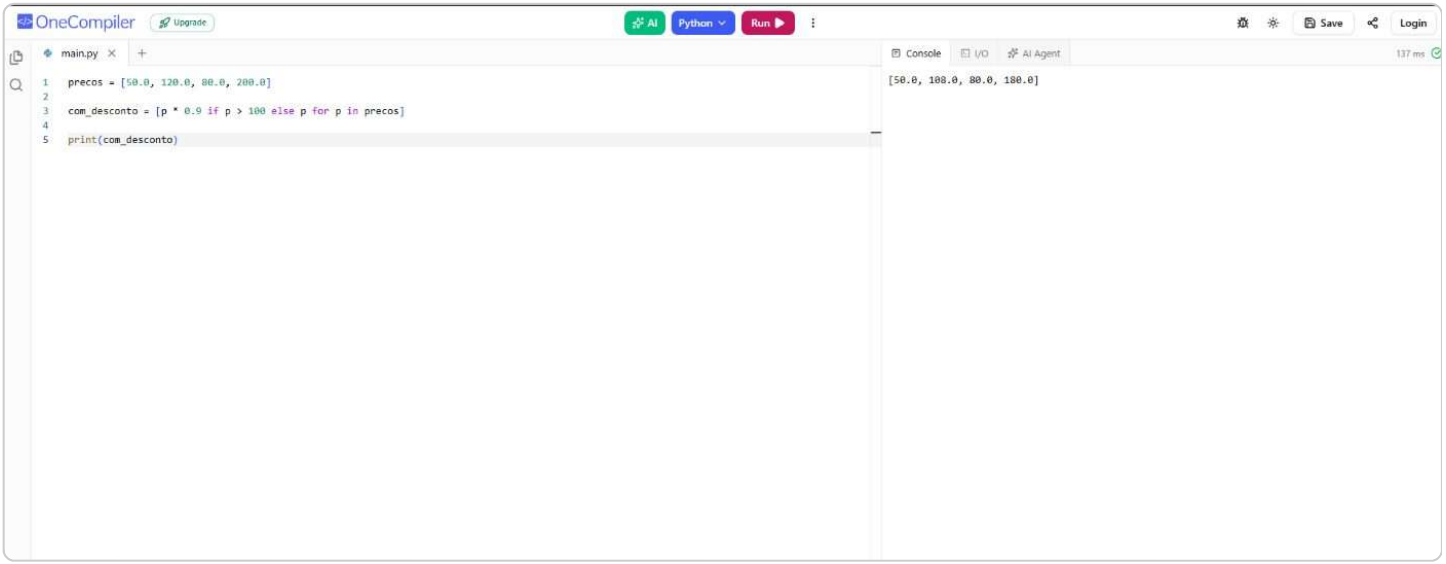
com_desconto = [p * 0.9 if p > 100 else p for p in precos]

print(com_desconto)
```

**Saídas das suas versões**

*não respondido*




**7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?**

Achei a versão funcional mais fácil de ler, pois o código fica mais curto e direto, já a versão com laço é mais fácil de depurar.

## Fechamento: O que fica do paradigma funcional

**Uma função pura pode ler uma variável global que muda.**

Falso

**Em Scheme, a recursão em cauda não faz a pilha crescer.**

Verdadeiro

**O Python otimiza a recursão em cauda.**

Falso

**Em Haskell, [1 ..] só é calculada à medida que é usada.**

Verdadeiro

**filter, map e reduce dispensam laço e variável acumuladora.**

Verdadeiro

**Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.**

Verdadeiro

**8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?**

Achei a versão em Haskell mais fácil de entender, pois é mais curta e mostra diretamente o que será feito. Aceitaria um desempenho menor em sistemas administrativos, onde a manutenção é mais importante. Já em sistemas de tempo real ou que exigem alto desempenho, priorizaria a velocidade.

**8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

Perceber que é possível trabalhar com listas infinitas no Haskell e executar apenas os cálculos necessários.

**8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?**

*não respondido*